Grounded or Guessed: A Reproducibility Study of LLM-Based Semantic Fault Localization

Doãn Văn An, Tạ Công Nam, Nguyễn Hoàng Hải, Đỗ Xuân Tráng,
Hoàng Trung Kiên, Mai Văn Tùng, Nguyễn Đức Tùng
School of Information and Communication Technology
Hanoi University of Science and Technology, Hanoi, Vietnam

***Abstract*—When tests fail, developers increasingly hand the failure to an LLM and ask where the bug is. Research tools propose far more elaborate pipelines that claim to ground the model's reasoning in program execution. We test that claim on SemLoc, a recent localizer for semantic faults that turns LLM-inferred constraints into a violation spectrum, and compare it with the everyday practice. We recompute every published configuration from the authors' logs, audit the evaluation, check the benchmark's labels by execution and by review, rerun everything three times with a current model, and ablate each signal. Four findings stand out. First, SemLoc's top-1 answer is guessed by design: an undocumented bonus always ranks the LLM's own guess first, and the counterfactual step never changes a ranking. Second, 112 of the benchmark's 250 fault labels move under review (56 sit on blank lines), and on the reviewed labels SemLoc's Acc@3 lead over direct prompting vanishes. Third, a single prompt with the failing tests beats the pipeline at Acc@1 on all three label sets (+10.2 points on execution-repaired labels) at about a seventh of the tokens; ablation shows the failing tests, not per-line coverage, carry the signal. Fourth, nothing more elaborate closes the gap—adding coverage to SemLoc helps it only slightly, and verifying candidate lines with test-passing patches, voting, or static analysis with ruff and pyrefly do not help at all—because small patches pass all tests in 23.3% of fix attempts at non-faulty candidate lines (dev split): the tests suggest where the bug is but cannot prove it. Code, data, LLM logs and a replayable cache are public.**

I. Introduction

Fault localization tells a developer where a failing program is wrong. In everyday practice the trigger is a failing test—run locally, in a pre-commit hook or on continuous integration—and, increasingly, the next step is to paste the failure into an AI assistant: most developers now use or plan to use such assistants [1], [2], debugging is among the tasks they bring to them [3], and error information measurably helps LLMs localize faults [4]. With test-driven development [5], those tests are written first and state what the code is meant to do. Linters and type checkers do not help at this point when the fault is in the logic: the code is well formed and well typed; it just computes the wrong value.

Research tools take a more elaborate route. Spectrum-based fault localization (SBFL) [6], [7], [8] ranks statements by how often failing and passing tests execute them, but it collapses on *semantic* faults, where both kinds of test run the same lines. SemLoc [9] targets exactly these faults: it asks an LLM for executable semantic constraints, instruments the program, runs the tests, ranks lines by a constraint-violation spectrum, and patches suspicious lines to verify them. Its promise is *structured grounding*: the model proposes, execution decides. It reports 42.8% Acc@1 on its SemFault-250 benchmark, far above SBFL.

This paper asks whether that accuracy is grounded or guessed, and whether the pipeline beats the everyday practice. Answering requires more than rerunning the tool: one must check how results are scored, whether the benchmark's labels point at the faults, how much each signal contributes, and whether conclusions survive real bugs, renamed code and other models. We study **RQ1** (reproduction and evaluation audit, including the labels), **RQ2** (what the LLM actually needs: a single prompt with failing tests, ablations, and more elaborate additions) and **RQ3** (robustness on BugsInPy [10], renamed programs, five model families and three label sets). Every LLM configuration runs three times, with program- and template-cluster bootstrap confidence intervals.

The answers are short. SemLoc's first line is always the LLM's guess, because the released code adds a bonus larger than any spectrum score; removing it changes Acc@1 by -9.1 points. Nearly half of the benchmark's labels do not point at the faulty line. And one prompt with the failing tests—close to what surveys suggest many developers do—beats the pipeline at Acc@1 on all three label sets for 2.0k instead of 14.0k tokens per program, while no more elaborate variant we tried closes the gap.

Contributions.

- **C1, guessed by design:** SemLoc's top-1 line is, by an undocumented choice, the LLM's own guess; its execution evidence acts below the first line; its counterfactual step (42% of LLM calls) never changes a ranking. We also report a released scoring script that credits delta debugging with 38.0% where the paper reports zero, two bugs in the released BugsInPy runner, and a re-execution that falls short of the logs.
- **C2, mislabeled benchmark:** 112 of 250 labels move under review; we release an execution-repaired label set (tests decide) and a reviewed one (an LLM pre-review, then team review), and show that on the reviewed labels SemLoc's Acc@3 lead over direct LLM prompting vanishes (it remains on the repaired labels).
- **C3, the simple prompt wins:** one prompt with failing tests beats SemLoc at Acc@1 by +10.2 [+5.3, +16.0] and

at Acc@3 by +14.5 points on repaired labels, at about a seventh of the cost.
- **C4, nothing more elaborate closes the gap, and why:** failing tests carry the signal and per-line coverage does not; CovGate (coverage added to SemLoc) helps SemLoc only slightly, and propose-and-verify (test-passing patches as arbiter), voting (both on the dev split) and static analysis do not improve on the simple prompt, because small patches pass the tests at many non-faulty candidate lines.
- **C5:** a seven-item evaluation checklist and a fully replayable artifact, including every reviewer's answers.

Code, data and LLM logs: https://github.com/doanan293/grounded-or-guessed.

II. Related Work and Research Direction

Spectrum- and mutation-based localization. Tarantula [6], Ochiai [7] and DStar [11] score statements from coverage; surveys [8], [12] note two evaluation pitfalls we revisit: tie handling and omission faults, which have no faulty line to blame. Mutation-based localization [13], [14] replaces coverage with mutant behavior; delta debugging [15] and cause-effect chains [16] underlie counterfactual reasoning; and Daikon [17] infers likely invariants from traces, whereas SemLoc asks an LLM for them.

LLM-based localization. AutoFL [18] lets an LLM navigate code with tool calls at method level; AgentFL [19] scales this to projects with agents; FlexFL [20] feeds SBFL output to open-source LLMs, the closest idea to our coverage baseline but at method level on Java. An industrial study of crash reports [21] finds that mined rankings in the prompt help LLMs localize, and EffiHolmes [22] contrasts two executions (normal vs. scaled workload) before LLM reasoning for performance faults. SemLoc [9] is the first to rank lines by violations of LLM-inferred executable constraints. Wu et al. [4] find that error logs markedly improve ChatGPT's localization, which works at function level but degrades at class level. None of these works measures how much of the accuracy comes from execution evidence rather than from the model's own guess, or compares with the plain practice of handing the failing tests to the model. For repository-level repair, Agentless [23] shows that a simple two-phase process beats more elaborate agents at lower cost.

Practice. Surveys report broad use of AI assistants by developers [1], [2] and DevGPT [3] collects real developer–ChatGPT conversations, debugging among them. Parnin and Orso [24] showed that automated debugging rests on assumptions that do not match how developers debug, and Kochhar et al. [25] surveyed practitioners' expectations of fault localization tools.

Benchmarks and validity. Defects4J [26], BugsInPy [10] and QuixBugs [27] provide faults with triggering tests. Pearson et al. [28] show that results on artificial faults do not transfer to real ones and that tie and omission handling can reorder techniques. For LLMs, threats include closed-model drift and benchmark contamination [29], [30], [31], mitigated by rewriting or renaming. Label errors are common even in flagship benchmarks and can reorder models [32]; Agentless also filters problematic SWE-bench tasks [23].

Plausible patches. In program repair, a patch that passes all tests is only *plausible*: most reported patches of early generate-and-validate systems were incorrect [33], and repair overfits weak test suites [34]. We meet the same effect from the localization side.

Direction. We combine these threads: we audit scoring as Pearson et al. recommend, check the benchmark's labels, test memorization by renaming, measure the share of accuracy that is grounded, and make the developers' plain practice the baseline every pipeline must beat.

III. Background

SBFL and metrics. For a line s, let e_f, e_p be the numbers of failing and passing tests that execute it and n_f the failing tests that do not. Ochiai scores s as $e_f/\sqrt{(e_f+n_f)(e_f+e_p)}$; lines with $e_f = 0$ are unranked. The rank of a program's fault is the best rank of any ground-truth line, and Acc@k is the fraction of programs whose fault ranks within k. We break ties by the *average* rank of the tied group and keep the denominator fixed: a program a tool fails on counts as a miss.

Semantic faults. We label a fault *semantic* when some faulty line is executed by at least one failing and one passing test (coverage cannot separate it), *coverage-revealed* when only failing tests reach it, and *not executed* when no failing test reaches any ground-truth line. Labels are recomputed for each ground-truth variant.

SemLoc. Fig. 1 summarizes SemLoc [9]. One LLM call returns constraints in a closed intermediate language, each attached to a program point; instrumentation evaluates them while the tests run; each constraint gets an Ochiai score over the constraint-by-test violation matrix, and a line gets the maximum score of the constraints attributed to it. Counterfactual verification drops constraints violated on passing tests, asks the LLM for a one-line patch at each remaining constraint's line, and re-runs the tests. The released code also adds 1.2 to the line the LLM names as its own fault-line guess—more than the maximum constraint score of one—which the paper does not describe.

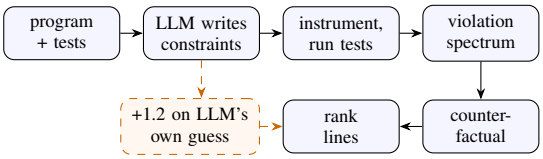


Fig. 1. SemLoc's pipeline [9]; the dashed box is the undocumented guess bonus found in the code.

IV. Approach

A. Evaluation audit

From the authors' logs we recompute every published configuration under two scorings: the *authors' scoring* as implemented in the released script (line-order tie breaking, any delta-debugging hit counted as rank one, programs that fail to run dropped from the denominator) and our *fair scoring* (average-rank ties, fixed denominator). We then remove the undocumented guess bonus, check whether the counterfactual step changes any ranking, and re-execute SemLoc with the paper's own model (Claude Sonnet 4.6) on the dev split.

B. Three label sets

Repaired (tests decide). 56 of the 250 ground-truth lines are blank and 65 programs have no failing test reaching the fault line. We re-derive fault locations with a *minimal-fix oracle*: a line is a fault location if replacing it (or filling a blank line, for omissions) with a small edit makes every test pass. An edit is small if its token-level edit distance to the original line (tokens are identifiers, numbers and single punctuation characters) is at most two or at most a third of the line's tokens. For each program: (1) for each ground-truth line, an LLM proposes candidate edits (GPT-6 luna, two attempts at $T = 0.8$, then up to four; a second pass with Claude Sonnet 5.5 for lines still unverified) and the tests decide; (2) a ground-truth line that admits a passing small edit is *confirmed*; (3) only if no failing test reaches the original line, lines within four of it are searched and the closest line with the smallest passing edit replaces it (*corrected*); (4) otherwise the line stays *unverified*. The oracle confirms 82 locations and corrects 46; 111 reachable lines keep their original labels (no proposed edit passed), and 11 programs cannot be verified and are dropped. Fixes written onto blank lines name no rankable statement and are also dropped, leaving 226 programs. Because executed labels are never searched for alternatives, the oracle cannot detect two equivalent fix locations.

Reviewed (LLM pre-review, team review). To check *every* label, an LLM (Claude Fable 5.1, high reasoning) read each program, its tests and its label, and judged the label right, wrong (proposing a line) or one of two lines of a two-line fault; it ran the tests on its own fixes for many cases. Then 6 team members reviewed 111 labels—the 46 the oracle had moved and every label the LLM judged wrong—with links to the code and tests, the candidate lines and the LLM's verdict as a suggestion; they agreed with the suggestion in 111 of 111 cases. The first author overrode 5 cases on test evidence (two-line faults that one-line fixes cannot repair, omission faults placed on the `return` statement as in sibling programs, and one mis-numbered LLM citation). The result covers all 250 programs, none on a blank line, with 44 two-line labels.

The 40 two-line extensions and the labels the LLM judged right rest on the LLM pre-review alone; no human reviewed them in the first round.

Blind re-review. To check the reviewed labels without the LLM's influence, 60 programs were sampled with a fixed seed (15 relocated by the oracle, 15 the LLM judged wrong, 20 it kept, 10 two-line) and each was judged independently by two members who had not seen it before. Candidate lines—the union of the original, oracle, LLM and reviewed lines, padded with neighbouring code lines to at least four—were lettered in random order, with no source and no LLM text. Sample and assignment were fixed in code before any answer. 36 of 120 answers cite the earlier, non-blind review files in their notes; they are excluded from the blind statistics by a rule recorded in code before computing agreement.

Use. The repaired labels involve no LLM judgment of location (an LLM proposes edits, the tests decide) and carry the main results; the reviewed labels are complete but LLM-suggested, which may favor LLM-based methods, so we use them as a sensitivity check. Every main result is also given on the original labels.

C. Grounded-vs-guessed decomposition

SemLoc's line score is the spectrum score plus a bonus of 1.2 on the LLM's guessed line. We define a two-player game {guess, spectrum}: $v(S)$ is Acc@k when lines are ranked by the signals in S only ($v(\emptyset) = 0$; "guess only" ranks the guessed line alone; "spectrum only" ranks by spectrum score with average-rank ties); with both, the scores equal SemLoc as released. We report the marginal values $v(\{\text{guess}\})$, $v(\{\text{spectrum}\})$ and $v(\{\text{guess}, \text{spectrum}\})$ together with the Shapley split. Because Acc@k is a mean of per-program hits, the aggregate Shapley value is the mean of per-program values, which lets us bootstrap confidence intervals by program and by template cluster. An extended game adds raw Ochiai (added with weight one) and counterfactual re-ranking as two more players; its grand coalition is a different system from SemLoc.

D. Methods compared

LLM + failing tests (ours). One call at $T = 0.8$ whose prompt gives the line-numbered program, the numbers of failing and passing tests, the statement that the program has exactly one faulty line, and each failing test's name with its first error line; the model returns its five most suspicious lines as JSON. This is the developers' practice made reproducible. The prompt was fixed before looking at test results.

Coverage ablation. *LLM + failing tests + coverage* is the same prompt with every line annotated `[f=`e_f`/p=`e_p`]` (and one sentence explaining the annotation). Against *LLM direct*—SemLoc's own direct-prediction mode, which sees the program only—the two paired contrasts separate the effect of the failing tests from that of coverage.

CovGate post-processes SemLoc's line scores with coverage. Writing $\sigma(s)$ for the spectrum score, g for the guessed line and F for the lines executed by some failing test,

$$c(s) = \mathbb{1}[s \in F]^{\gamma} \left(\sigma(s) + \beta \mathbb{1}[s = g] + \lambda \,\mathrm{Ochiai}(s)\right), \quad (1)$$

where $\gamma \in \{0, 1\}$ switches a coverage gate, $\lambda \in \{0; 0.01; 0.1; 0.5\}$ weights Ochiai, and the bonus β is either always 1.2 or kept only when execution supports g; a fourth switch promotes lines confirmed by the counterfactual patches. Selected on the dev split, CovGate has no gate and the original bonus, with λ = 0.01 (original) or 0.5 (repaired labels): an Ochiai tie-breaker.

Propose-and-verify uses the tests as arbiter: LLM + failing tests proposes lines; for the first k, the LLM writes a small fix at that line (the oracle's proposer and size criterion), the patched program runs against the tests, and lines with a passing fix move to the front. A 32-point grid (k, attempts per line, order among verified lines, maximum fix size) is evaluated offline from one widest run on the dev split, three repeats; the selection rule (mean dev Acc@1 over the original and reviewed labels; the repaired labels are excluded because they were built by the same fix-and-test mechanism) was fixed in code before running.

Voting merges three answers of LLM + failing tests by summed reciprocal rank (dev split).

Static analysis. Pinned versions of ruff (default and all rules) and pyrefly (type checking) run on all 250 programs; a program counts as hit when a diagnostic sits on a labeled line, compared with the hits expected if the flagged lines were spread at random over the program's code lines.

V. Evaluation

A. Setup

SemFault-250 [9] has 250 single-function Python programs with one injected fault and about ten tests each. We fix a 50/200 dev/test split stratified by fault label and group structurally similar programs into template clusters (4-gram Jaccard similarity of at least four fifths on normalized code). Real bugs are the 31 youtube-dl bugs of BugsInPy [10] whose environment builds, whose triggering tests fail before and pass after the fix, and whose coverage can be measured; the target function is the one containing a developer-changed line that a declared failing test executes, and its changed lines are the ground truth. The renamed variant replaces every program-defined identifier with an opaque name and blanks comments while preserving line numbers and each test's outcome (199 test programs). The main model is Gemini 3.8 Flash; the model-family study adds GPT-6 luna, Claude Sonnet 5.5, gpt-oss-120b and Gemini 3.5 Flash-Lite on a fixed subset of the test split (44 programs with repaired labels). All models run at their provider-default reasoning level; SemLoc and the single-prompt methods run at $T = 0.8$. Every LLM configuration is repeated three times; a run is used only if it finished without unrecovered LLM errors. Differences are Δ (percentage points) with 95% bootstrap CIs that resample template clusters (10000 draws) after averaging each program over repeats; we also report each repeat's McNemar test and a matched-pairs rank-biserial effect size on reciprocal rank (artifact). We report about thirty such comparisons without multiplicity correction and read single intervals near zero with caution.

TABLE I
Published configurations under the authors' and fair scoring (released logs, 250 programs)

	Author scoring			Fair scoring		
Configuration (all 250 programs)	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3	Acc@5
SemLoc, Claude Sonnet 4.6, T=0.8	42.8	68.0	76.8	42.8	65.6	76.8
SemLoc, Gemini 2.5 Pro, T=0.8	39.8	64.7	73.5	39.6	64.4	72.8
SemLoc Claude T=0.8 without LLM-guess bonus	–	–	–	30.0	55.6	66.4
SemLoc Gemini T=0.8 without LLM-guess bonus	–	–	–	27.2	55.6	64.0
SemLoc w/o CF (separate run), Claude	30.8	59.6	68.8	–	–	–
LLM direct, Claude	35.2	41.2	41.2	35.2	41.2	41.2
SBFL Ochiai	6.4	13.2	30.4	4.0	16.0	48.0
SBFL DStar (recomputed)	–	–	–	4.0	16.0	48.0
Delta debugging	38.0	38.0	38.0	0.0	0.0	0.0

B. RQ1: Reproduction and evaluation audit

Recomputing from the released logs reproduces all 13 published configurations exactly. Re-executing SemLoc with the paper's own model on the dev split does not: Acc@1 32.0% and Acc@3 44.0% against 36.0% and 68.0% in the logs for the same programs; the re-executed constraints are violated in failing tests less often, which we could not trace to our infrastructure.

Table I shows how scoring matters. The released scoring script credits delta debugging with 38.0% (any hit counts as rank one), although the paper reports zero and fair scoring gives 0.0%, since it ranks no line; SBFL's Acc@5 rises from 30.4% to 48.0% under average-rank ties. SemLoc is stable under fair scoring, but without the undocumented guess bonus its Acc@1 falls from 42.8% to 30.0% (Claude) and from 39.6% to 27.2% (Gemini). In the released code the counterfactual step never changes a ranking (0 of 250 programs in all five configurations with data). The paper's w/o-counterfactual figure comes from a separate run whose scores were not released; its Acc@1 (30.8%) is close to SemLoc without the bonus, which suggests a configuration difference rather than an effect of the step, but we cannot determine which. The released BugsInPy runner reports the rank of the LLM's own guess rather than of the fixed lines, and its patch-and-restore step leaves stale bytecode that can hide the bug in the next run; we fixed the latter in our driver.

Labels. Table II compares the original labels with the reviewed ones: only 98 are unchanged, 40 keep the line and gain a second one, and 112 move—56 from a blank line, 7 from a log call, `def` header or comment, 32 by one line and 17 by more. Moves follow patterns: a label on a guard whose guarded statement holds the fault (or vice versa), or an omission fault labeled at its insertion point, a blank line. We suspect a labeling tool rather than human annotators but cannot check without the generator. In the blind re-review, the two reviewers chose the same line in 37 of 42 fully blind pairs, and 71 of 84 blind answers match the reviewed label against 38 for the original label. Every blind answer on relocated or LLM-rejected labels picks the reviewed line (22/22 and

TABLE II
ORIGINAL VS. REVIEWED LABELS (ALL 250 PROGRAMS)

Original label vs. reviewed label ($n = 250$)	Programs
Same line	98
Kept, second line added (two-line fault)	40
Blank line	56
Log call, `def` header or comment	7
One line away	32
Two or more lines away	17
Moved (blank, non-code, one or more lines away)	112

6/6; few relocated answers stayed blind); two-line labels reach 18/20. The hardest group is the labels the LLM kept (25/36): every mismatch comes with a note that no bug was visible—omission faults (a missing normalization or remainder step) with no wrong-looking token. The blind review thus supports the relocations and shows that omissions are where line labels are inherently ambiguous. Mislabels lower every number, and they may favor SemLoc: on the original labels SemLoc beats LLM direct at Acc@3 (60.3% vs. 43.0%); on the reviewed labels the two tie (90.0% vs. 90.8%) and LLM direct leads at Acc@1 (+4.3 [+0.3, +8.6]). On the repaired labels, which involve no LLM judgment of location, SemLoc's lead remains (LLM direct ΔAcc@3 -12.7 [-19.7, -6.2]), so the vanishing lead rests on the LLM-suggested labels and must be read with the circularity caveat of Section VII. A plausible reason for the effect: LLM direct names one or two lines, whereas SemLoc ranks many adjacent lines and so still "hits" a label one line off.

C. RQ2: What does the LLM need?

The simple prompt wins. Table III gives the main comparison. LLM + failing tests beats SemLoc at Acc@1 on all three label sets: +7.7 [+3.5, +12.4] (original), +10.2 [+5.3, +16.0] (repaired) and +12.5 [+8.0, +17.5] (reviewed). At Acc@3 it gains +14.5 [+8.5, +20.8] (repaired) and +10.0 [+6.3, +14.3] (reviewed); over all original labels, which include faults no failing test reaches, the Acc@3 gain (+7.2) has a CI touching zero, while on reached faults it is +13.7 [+6.9, +20.4]. It costs 2.0k tokens per program against SemLoc's 14.0k. On the reviewed labels it reaches 100.0% Acc@3: with reviewed (LLM-suggested) labels, the benchmark is nearly saturated by one prompt.

What carries the signal. The failing tests do: against LLM direct they change Acc@3 by +27.2 [+18.6, +37.0] points (repaired). Per-line coverage does not: adding it changes Acc@3 by +2.1 [-0.2, +4.9] and Acc@1 by -5.4 [-9.4, -2.1] (repaired), and Acc@1 by -3.3 on the reviewed labels. The model needs to know which test fails and how, not which lines ran.

More elaborate additions do not help (Table IV). CovGate, an Ochiai tie-breaker as selected, changes SemLoc's Acc@3 by +3.9 [+1.6, +6.8] and leaves Acc@1 unchanged; its other components lower Acc@1 (grounded bonus -4.1, counterfactual re-ranking -2.4). For propose-and-verify, the selected configuration reproduces the proposer's ranking exactly (reviewed Acc@1 90.7% vs. 90.7% on dev), and the most aggressive one drops it to 79.3%. The tests are the reason: on the reviewed labels a small fix passes every test in 42.5% of attempts at the faulty line (two-line faults, omissions and larger edits resist one-line fixes) but also in 23.3% of attempts at other lines, typically by changing a constant to compensate for a wrong computation elsewhere. Since the dev result was clear, we did not run it on the test split. Voting over three answers (dev) gives Acc@1 92.0% against 90.7% for one answer on the reviewed labels, but lowers Acc@3 on the repaired (80.8% vs. 85.1%) and original labels (56.0% vs. 60.0%).

Static analysis (Table V). Ruff's default rules flag a labeled line in 1 of 250 programs. With all rules it emits 4172 diagnostics and hits 25 programs, *fewer* than the 56.3 expected at random: style warnings cluster on loops and long conditionals, whereas faults sit in short comparisons and arithmetic. Pyrefly reports 21 type errors, none on a labeled line; some are symptoms a few lines below (a float returned where an int is declared). These tools check how the code is written, not what it is meant to do.

Grounded or guessed. Because the bonus exceeds any spectrum score, adding the spectrum to the guess cannot change the first line: guess only, and SemLoc, both reach 52.3% Acc@1 (repaired), while spectrum only reaches 43.2%; on the reviewed labels the figures are 83.0% and 56.8%. The Shapley value of the spectrum at Acc@1 (21.6, Table VI) is credit for what the spectrum would achieve without the guess, not a contribution to SemLoc's top line. At Acc@3 the spectrum matters—the guess stays at 52.3%, together they reach 75.4%—but the single prompt reaches higher without it. On the authors' logs (Claude) the split is 42.8 = 27.8 + 15.0.

Cost across models. Fig. 2 places five models on one subset using the single prompt with coverage, the variant run on every model: for every model it has higher Acc@3 at lower cost than SemLoc. These per-model comparisons have no CIs; at Acc@1 GPT-6 luna does not improve.

D. RQ3: Robustness

Real bugs. On the 31 youtube-dl bugs (Table VII), SemLoc's top-1 is the guess by construction, and the guess's Acc@1 (46.2%) is SemLoc's; the spectrum alone places 7.5% first, close to SBFL (9.7%). LLM + failing tests points the same way as on SemFault-250 (Acc@3 71.0% vs. 53.8%) and coverage again adds nothing (69.9%), but with 31 bugs from one project the CI [-3.2, +36.6] includes zero. On real bugs every method is far from the ceiling. "Fault reached" keeps the bugs where a failing test executes a labeled line; "artifact's own metric" is the rank the released runner reports, i.e., that of the LLM's guess.

Renamed programs. Pairing each program with its renamed copy (Table VIII; Gemini 3.8 Flash only), the guess shows no detectable change (Acc@1 Δ -2.6 [-7.8, +2.4]), so we see no sign that the model guesses from memorized names. The constraint spectrum needs meaningful names: without the bonus, Acc@3 changes by -15.5 points. The single prompt (with coverage; the variant without coverage was not run on

TABLE III
MAIN RESULTS ON THE TEST SPLIT UNDER THREE LABEL SETS (GEMINI 3.8 FLASH, MEAN OF THREE REPEATS) AND LLM TOKENS PER PROGRAM

	Original ($n = 200$)		Repaired ($n = 179$)		Reviewed ($n = 200$)		
Method	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3	k tokens
SBFL Ochiai	3.0	15.5	3.9	19.0	11.0	28.0	0
LLM direct (source only)	40.0	43.0	56.8	62.8	87.3	90.8	2.5
SemLoc (multi-stage)	36.8	60.3	52.3	75.4	83.0	90.0	14.0
LLM + failing tests (ours)	44.5	67.5	62.6	89.9	95.5	100.0	2.0
+ per-line coverage	40.2	68.2	57.2	92.0	92.2	100.0	2.6

TABLE IV
ADDITIONS COMPARED WITH THE METHOD THEY EXTEND: Δ IN POINTS (TEST ROWS: THREE REPEATS WITH 95% CLUSTER-BOOTSTRAP CIS; DEV ROWS: 50 PROGRAMS, THREE REPEATS)

		Repaired		Reviewed	
Addition (compared with)	Split	ΔAcc@1	ΔAcc@3	ΔAcc@1	ΔAcc@3
Per-line coverage (LLM + failing tests)	test	-5.4 [-9.4, -2.1]	+2.1 [-0.2, 4.9]	-3.3 [-5.9, -1.2]	+0.0 [0.0, 0.0]
CovGate (SemLoc)	test	+0.0 [0.0, 0.0]	+3.9 [1.6, 6.8]	+0.0 [0.0, 0.0]	+1.2 [0.0, 2.8]
Propose-and-verify, selected (LLM + failing tests)	dev	+0.0	+0.0	+0.0	+0.0
Propose-and-verify, most aggressive (LLM + failing tests)	dev	-4.3	+0.0	-11.3	+0.0
Vote over 3 answers (LLM + failing tests)	dev	+0.0	-4.3	+1.3	+0.7

TABLE V
STATIC ANALYSIS ON THE 250 PROGRAMS (REVIEWED LABELS)

Tool	Diagnostics	Programs flagged	On a fault line	Expected by chance
ruff (default rules)	148	109	1	2.4
ruff (all rules)	4172	250	25	56.3
pyrefly (type check)	21	21	0	0.6

TABLE VI
SHAPLEY SPLIT OF SEMLOC'S ACCURACY (OUR RUNS, THREE REPEATS; TEMPLATE-CLUSTER CIS). UPPER BLOCK: THE GAME WHOSE TOTAL IS SEMLOC AS RELEASED; LOWER BLOCK: ALL FOUR SIGNALS (A DIFFERENT SYSTEM).

Ground truth	Metric	Total	LLM guess	Semantic spectrum	Coverage	CF
SemLoc as released (guess + spectrum)						
Original ($n = 200$)	Acc@1	36.8	22.2 [16.5, 28.4]	14.6 [11.8, 17.5]	–	–
Original ($n = 200$)	Acc@3	60.3	19.8 [16.1, 23.8]	40.6 [35.6, 45.3]	–	–
Repaired ($n = 179$)	Acc@1	52.3	30.7 [23.5, 37.8]	21.6 [17.4, 26.1]	–	–
Repaired ($n = 179$)	Acc@3	75.4	28.5 [23.4, 33.5]	46.9 [41.9, 51.7]	–	–
All signals (+ raw Ochiai, + CF re-ranking)						
Original ($n = 200$)	Acc@1	35.0	22.7 [17.4, 28.3]	12.5 [10.0, 15.2]	-0.5 [-1.5, 0.6]	0.3 [-0.6, 1.1]
Original ($n = 200$)	Acc@3	59.0	17.6 [14.2, 21.4]	35.8 [31.6, 40.2]	4.6 [0.1, 9.0]	1.1 [0.5, 1.8]
Repaired ($n = 179$)	Acc@1	49.9	31.9 [25.3, 38.5]	18.6 [14.4, 22.9]	-1.1 [-2.3, 0.2]	0.5 [-0.7, 1.5]
Repaired ($n = 179$)	Acc@3	80.6	25.9 [20.8, 31.0]	44.4 [39.7, 49.3]	9.4 [5.8, 13.4]	0.9 [0.3, 1.7]

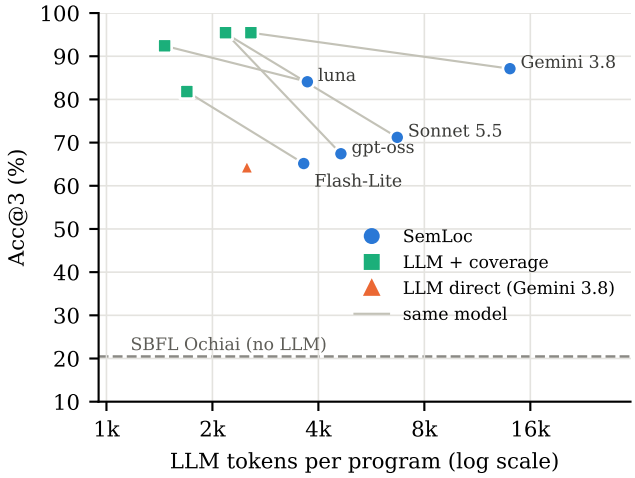


Fig. 2. Cost vs. Acc@3 on the same subset (44 programs, repaired labels); lines join the two methods of one model.

renamed programs) shows no detectable change (Acc@3 Δ -0.9 [-3.5, +1.3]). Renaming leaves test names intact, so they still state the intent.

Model families. On the subset (Table IX; no CIs), weaker models lean harder on the guess: without the bonus, Flash-Lite's Acc@1 falls from 42.4% to 20.4% and gpt-oss-120b's from 45.5% to 29.5%; the subset is too small to rank models on this effect.

Label sets. On all three label sets the single prompt beats SemLoc at Acc@1, coverage adds nothing and removing the bonus lowers SemLoc's Acc@1 (Tables III and IV). The only comparison that flips with the labels is SemLoc vs. LLM direct at Acc@3, and only on the LLM-suggested reviewed labels.

E. Failure analysis

We categorize every test program automatically (Table X) and read a cluster-spread sample of up to four failures per category, nineteen in total.

The guess is right for 94 of 179 programs; the spectrum rescues 25 wrong guesses into the top three. Wrong guesses are rarely far off: over the 527 program–repeat pairs with a guess (10 runs returned none), the guess hits the fault line 281 times, lands one line below it 99 times and one line above only 28 times. In 44 of 85 wrongly guessed programs, guess and label are a compound-statement header and the statement it guards, as here, where the injected fault is the `- 1`:

TABLE VII
RQ3A: 31 YOUTUBE-DL BUGS (BUGSINPY), THREE REPEATS

	All bugs ($n = 31$)			Fault reached ($n = 26$)	
Method	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3
SemLoc semantic spectrum only	7.5±1.9	29.0±3.2	37.6±3.7	9.0±2.2	34.6±3.9
SemLoc (spectrum + LLM-guess bonus)	46.2±6.7	53.8±7.4	55.9±6.7	51.3±8.0	60.3±8.9
LLM guess alone	46.2±6.7	46.2±6.7	46.2±6.7	51.3±8.0	51.3±8.0
CovGate (ours)	46.2±6.7	54.8±6.5	57.0±7.4	51.3±8.0	61.5±7.7
LLM + failing tests (1 prompt)	50.5±1.9	71.0±3.2	77.4±3.2	56.4±2.2	71.8±2.2
LLM + coverage (1 prompt, ours)	51.6±5.6	69.9±3.7	75.3±1.9	59.0±5.9	75.6±4.4
SBFL Ochiai	9.7±0.0	29.0±0.0	29.0±0.0	11.5±0.0	34.6±0.0
SBFL DStar	9.7±0.0	29.0±0.0	29.0±0.0	11.5±0.0	34.6±0.0
Artifact's own metric (rank of the LLM guess)	39.8±1.9	45.2±3.2	51.6±3.2	37.2±2.2	43.6±5.9

TABLE VIII
RQ3B: ORIGINAL VS. IDENTIFIER-RENAMED PROGRAMS (REPAIRED LABELS, THREE PAIRED REPEATS)

	Acc@1			Acc@3		
Method	Original	Renamed	Δ [95% CI]	Original	Renamed	Δ [95% CI]
SemLoc	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]	75.3±2.0	66.8±2.0	-8.4 [-14.1, -3.0]
w/o LLM-guess bonus	42.9±2.1	34.5±2.1	-8.4 [-13.5, -3.7]	70.6±2.3	55.1±2.6	-15.5 [-21.4, -10.3]
LLM guess alone	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]	52.1±0.9	49.4±1.0	-2.6 [-7.8, 2.4]
LLM + coverage (1 prompt)	56.9±0.9	56.2±0.0	-0.8 [-3.6, 2.0]	92.0±0.3	91.0±1.5	-0.9 [-3.5, 1.3]

TABLE IX
RQ3C: FIVE MODELS ON THE SAME SUBSET (REPAIRED LABELS, THREE REPEATS, NO CIS)

		SemLoc		w/o LLM-guess bonus		CovGate
Model	Repeats	Acc@1	Acc@3	Acc@1	Acc@3	Acc@3
Gemini 3.8 Flash	3	52.3±2.3	87.1±1.3	53.0±4.7	84.1±0.0	87.9±2.6
GPT-6 luna	3	53.8±2.6	84.1±2.3	40.9±6.0	72.7±2.3	84.9±1.3
Claude Sonnet 5.5	3	43.2±2.3	71.2±1.3	37.9±1.3	68.9±1.3	71.2±1.3
gpt-oss-120b	3	45.5±6.0	67.4±3.5	29.5±7.9	57.6±6.6	68.2±2.3
Gemini 3.5 Flash-Lite	3	42.4±3.5	65.1±5.7	20.4±2.3	51.5±4.7	71.2±1.3

TABLE X
PER-PROGRAM OUTCOME CATEGORIES (REPAIRED LABELS, MAJORITY OVER THREE REPEATS); FIRST MATCH WINS

Category	Definition	Programs
gt_suspect	unverified label, every method misses top-3	6
guess_right	the LLM's guess is a ground-truth line	94
prior_hurts	spectrum ranks the fault first, the bonus lifts a wrong guess above it	19
rescued_by_spectrum	wrong guess, fault still in SemLoc's top-3	25
coverage_only_wins	SemLoc misses top-3, the single prompt (with coverage) hits it	32
all_miss	everything else	3

```
if cap is not None and counters[key] > cap:  # label
    counters[key] = cap - 1                  # guess
```

We measure two tolerant variants: *strict* accepts the header and the first line of its body; the *upper bound* accepts the header and any line of its first body statement (up to three lines, nested guards included). SemLoc's Acc@1 rises from 52.3% to 73.9% (strict) or 76.2% (upper bound), and the single prompt's (with coverage) from 57.2% to 79.3%–81.6%; it stays ahead of SemLoc under every variant. Reading the sample adds three more patterns: multi-line injected blocks where either line fixes the bug, cause and symptom on one data-flow path, and fixes our oracle accepted that compensate rather than repair (e.g., subtracting one from `norm` instead of fixing `max_norm / (norm + 1)`). Most cases where the bonus "hurts" are of the first two kinds: the guess is semantically right and one line off.

VI. DISCUSSION

Reading "guessed" correctly. SemLoc's top line is the LLM's guess by design, and the failure analysis shows the guess is often semantically right and one line off the label. The lesson is therefore not that the bonus should go—on our runs removing it changes Acc@1 by -9.1 points—but that a tool which claims to be grounded should state that its first answer is the model's, and that line-level labels need to accept equivalent fix locations before small differences between methods mean much.

Why tests make poor arbiters of location. Three results share one cause. SemLoc's counterfactual step changes no ranking; our minimal-fix oracle moved 8 of its 46 relocated labels to lines the review rejected; and propose-and-verify does not improve on its proposer. All three accept a line when a small patch there makes every test pass. With about ten tests per program, that criterion holds in 23.3% of fix attempts at non-faulty candidate lines—usually by a compensating constant or subtraction—and in only 42.5% at faulty ones (dev split, the proposer's top candidates), which two-line faults, omissions and larger edits put out of reach. This is the plausible-patch problem [33], [34] seen from localization: the tests suffice to *suggest* where the bug is, not to *prove* it.

TABLE XI
CHECKLIST FOR EVALUATING LLM-BASED FAULT LOCALIZATION, WITH WHAT EACH CHECK CHANGED HERE

Check	Effect in this study
Average-rank ties	SBFL Acc@5 30.4% → 48.0%
Fixed denominator, any-hit not rank one	delta debugging 38.0% → 0.0% in the released script
Checked labels	112 of 250 labels move; SemLoc's Acc@3 lead over LLM direct vanishes on reviewed labels
Ablate every component, incl. undocumented	guess bonus: Acc@1 changes by -9.1 without it
Repeats with paired CIs	several single-repeat differences are not significant; pooled ones are
Report cost	counterfactual step: 42% of calls, no ranking change
Single prompt with failing tests	beats the pipeline at Acc@1 on all label sets (+10.2 repaired)

Saturation and practice. With the reviewed (LLM-suggested) labels one prompt reaches 100.0% Acc@3. SemFault-250's programs are single short functions, its faults are one-token mutations, and its test names state the violated intent (e.g., `test_fee_deducted_once_not_per_participant`); much of the difficulty the original paper measured came from mislabels. The benchmark can no longer show that an elaborate pipeline is needed; real bugs, where every method stays far from the ceiling, can. For practitioners, the strongest investment is tests that state intent, as test-driven development produces; and an AI agent that stops when the tests pass should have its patch reviewed, since about a quarter of fix attempts at plausible but non-faulty lines yield a test-passing patch on suites of this size.

An evaluation checklist. Table XI collects seven checks; each changed at least one number in this study.

VII. THREATS TO VALIDITY

Internal and construct validity. The minimal-fix oracle can accept compensating fixes and never searches alternatives for reachable labels; the review overturned 8 of its relocations. The reviewed labels are LLM-suggested; in the first round reviewers saw the suggestion and agreed in 111 of 111 cases. The blind re-review supports them, but on a sample of 60 programs, with 36 non-blind answers excluded and choices limited to the listed candidates, so the reviewed labels are still not fully independent of LLM judgment and may favor LLM-based methods; we therefore state main results on the original and repaired labels, which involve no LLM judgment of location. Two-line labels (44 programs) make the reviewed Acc@k slightly easier. Line-level Acc@1 depends on whether a guard or the guarded statement is labeled; we report tolerant variants. The single prompt is ours, fixed before seeing test results. Propose-and-verify and voting were evaluated on the dev split only; their negative result rests on a zero difference at the selected configuration and on the measured mechanism, not on the test split.

External validity. Test names in SemFault-250 state intent unusually clearly; on suites with generic names the single prompt's advantage may shrink, and we did not run a hidden-test-name ablation. Every program is one function, and for real bugs the target function is taken from the developer fix, so localizing the function within a project—the hard part on large code bases—is not evaluated. Real bugs come from one project (31 youtube-dl bugs). RQ2, the decomposition and the renaming study use one model; the model-family subset has no CIs; models run at provider-default reasoning. Renaming changes identifiers and comments but not structure or test names, so memorization of templates is not fully excluded.

VIII. CONCLUSION AND FUTURE WORK

SemLoc's published numbers recompute exactly from its logs, but its top-1 answer is, by an undocumented design choice, the LLM's own guess, and nearly half of its benchmark's labels do not point at the faulty line. Handing the failing tests to the model in one prompt—close to what surveys suggest many developers do—is more accurate at Acc@1 on all three label sets and about seven times cheaper; the failing tests carry the signal, per-line coverage does not, and no more elaborate addition we tried closes the gap (CovGate helps SemLoc only slightly; the others and static analysis do not help), because the tests can suggest a fault's location but cannot prove it. These results come from one main model, single-function programs with intent-revealing tests, and 31 real bugs from one project. We suggest that every evaluation of LLM-based fault localization apply the seven checks of Table XI, with the plain failing-tests prompt as a mandatory baseline. Future work includes project-level localization (narrowing a repository to a function before asking the model), a hidden-test-name ablation, stronger test suites that make execution a usable arbiter of location, and real bugs from more projects.

DATA AVAILABILITY

Code, derived data, run outputs, LLM call logs, a replayable LLM response cache, every reviewer's answers and an offline Docker image are at https://github.com/doanan293/grounded-or-guessed; `REPRODUCE.md` maps every table to its script.

REFERENCES

[1] Stack Overflow, "2025 developer survey," 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[2] JetBrains, "The state of developer ecosystem 2025," 2025. [Online]. Available: https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/

[3] T. Xiao, C. Treude, H. Hata, and K. Matsumoto, "DevGPT: Studying developer-ChatGPT conversations," in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR), Mining Challenge*, 2024. [Online]. Available: https://arxiv.org/abs/2309.03914

[4] Y. Wu, Z. Li, J. M. Zhang, M. Papadakis, M. Harman, and Y. Liu, "Large language models in fault localisation," 2023. [Online]. Available: https://arxiv.org/abs/2308.15276

[5] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.

[6] J. A. Jones, M. J. Harrold, and J. Stasko, "Visualization of test information to assist fault localization," in *Proceedings of the 24th international conference on Software engineering - ICSE '02*, ser. ICSE '02. ACM Press, 2002, p. 467. [Online]. Available: http://dx.doi.org/10.1145/581396.581397

[7] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, "A practical evaluation of spectrum-based fault localization," *Journal of Systems and Software*, vol. 82, no. 11, p. 1780–1792, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2009.06.035

[8] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, p. 707–740, Aug. 2016. [Online]. Available: http://dx.doi.org/10.1109/TSE.2016.2521368

[9] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu, "Semloc: Structured grounding of free-form llm reasoning for fault localization," 2026. [Online]. Available: https://arxiv.org/abs/2603.29109

[10] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, "Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '20. ACM, Nov. 2020, p. 1556–1560. [Online]. Available: http://dx.doi.org/10.1145/3368089.3417943

[11] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The dstar method for effective software fault localization," *IEEE Transactions on Reliability*, vol. 63, no. 1, p. 290–308, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1109/TR.2013.2285319

[12] H. A. de Souza, M. L. Chaim, and F. Kon, "Spectrum-based software fault localization: A survey of techniques, advances, and challenges," 2016. [Online]. Available: https://arxiv.org/abs/1607.04347

[13] M. Papadakis and Y. Le Traon, "Metallaxis-fl: mutation-based fault localization," *Software Testing, Verification and Reliability*, vol. 25, no. 5-7, p. 605–628, Sep. 2013. [Online]. Available: http://dx.doi.org/10.1002/stvr.1509

[14] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, Mar. 2014, p. 153–162. [Online]. Available: http://dx.doi.org/10.1109/ICST.2014.28

[15] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, p. 183–200, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1109/32.988498

[16] A. Zeller, "Isolating cause-effect chains from computer programs," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 6, p. 1–10, Nov. 2002. [Online]. Available: http://dx.doi.org/10.1145/605466.605468

[17] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, p. 35–45, Dec. 2007. [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015

[18] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization," 2023. [Online]. Available: https://arxiv.org/abs/2308.05487

[19] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "Agentfl: Scaling llm-based fault localization to project-level context," 2024. [Online]. Available: https://arxiv.org/abs/2403.16362

[20] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.10714

[21] M. Medeiros, U. Kulesza, C. Treude, D. Lucena, R. Gomes, R. Coelho, E. Adachi, and R. Bonifacio, "Integrating crash report mining and llms for bug localization and repair: An industrial report," 2026. [Online]. Available: https://arxiv.org/abs/2609.04483

[22] H. Yang, Y. Peng, and Z. Ding, "Effiholmes: Differential profiling-guided repository level time inefficiency fix localization," 2026. [Online]. Available: https://arxiv.org/abs/2608.03558

[23] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[24] C. Parnin and A. Orso, "Are automated debugging techniques actually helping programmers?" in *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2011, pp. 199–209.

[25] P. S. Kochhar, X. Xia, D. Lo, and S. Li, "Practitioners' expectations on automated fault localization," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 165–176.

[26] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA '14. ACM, 2014, p. 437–440. [Online]. Available: http://dx.doi.org/10.1145/2610384.2628055

[27] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH '17. ACM, Oct. 2017, p. 55–56. [Online]. Available: http://dx.doi.org/10.1145/3135932.3135941

[28] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, May 2017, p. 609–620. [Online]. Available: http://dx.doi.org/10.1109/ICSE.2017.62

[29] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: the threats of using llms in software engineering," 2023. [Online]. Available: https://arxiv.org/abs/2312.08055

[30] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, "Are large language models memorizing bug benchmarks?" 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[31] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, "Benchmark data contamination of large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2406.04244

[32] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2103.14749

[33] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2015, pp. 24–36.

[34] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 532–543.